



# Ringtail: Lattice-Based Post-Quantum Threshold Signatures for Blockchain Consensus

Zach Kelling

*Lux Industries*

2024

## Abstract

We present Ringtail, a lattice-based post-quantum threshold signature scheme designed for blockchain consensus. Ringtail enables  $t$ -of- $n$  validators to collaboratively sign blocks without any single party possessing the full secret key, while providing security against quantum adversaries. The scheme operates in two rounds: a precomputation round independent of the message, and a signing round that produces the final signature. Ringtail builds on the Module-LWE assumption over polynomial rings  $\mathbb{Z}_q[X]/(X^N + 1)$  with parameters  $N = 256$ ,  $q = 2^{48} + 2561$  (a 48-bit NTT-friendly prime), module dimensions  $M = 8 \times N = 7$ , and Shamir secret sharing over the ring. We prove that the scheme achieves existential unforgeability under chosen-message attacks (EUF-CMA) in the random oracle model, reducing security to the hardness of Module-LWE with parameters providing  $> 128$ -bit classical and  $> 100$ -bit quantum security. The signature consists of a challenge polynomial  $c \in \mathbb{Z}_q[X]/(X^N + 1)$  and a response vector  $\mathbf{z} \in (\mathbb{Z}_q[X]/(X^N + 1))^N$ , with total size approximately 56 KB for  $N = 256$ . Key generation uses Shamir secret sharing of a discrete Gaussian vector over the polynomial ring, distributing shares to  $n$  parties with reconstruction threshold  $t$ . The two-round signing protocol uses correlated randomness from PRF seeds shared between parties and MAC-based authentication to prevent malicious share injection. Verification requires one matrix-vector multiplication and one hash evaluation. We demonstrate 0.6-second signing latency across 5 geographically distributed parties (2-of-3 threshold) and integration with the Lux Quasar consensus engine for quantum-safe block finalization.

## Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>3</b> |
| <b>2</b> | <b>Preliminaries</b>                                  | <b>3</b> |
| 2.1      | Polynomial Rings . . . . .                            | 3        |
| 2.2      | Shamir Secret Sharing over Rings . . . . .            | 3        |
| 2.3      | Ringtail Parameters . . . . .                         | 4        |
| <b>3</b> | <b>The Ringtail Scheme</b>                            | <b>4</b> |
| 3.1      | Key Generation (Trusted Dealer) . . . . .             | 4        |
| 3.2      | Signing Protocol (2 Rounds) . . . . .                 | 5        |
| 3.3      | Finalization and Verification . . . . .               | 5        |
| <b>4</b> | <b>Security Analysis</b>                              | <b>5</b> |
| 4.1      | Unforgeability . . . . .                              | 5        |
| 4.2      | Threshold Security . . . . .                          | 6        |
| 4.3      | Concrete Security Estimates . . . . .                 | 6        |
| <b>5</b> | <b>Integration with Lux Quasar</b>                    | <b>7</b> |
| 5.1      | Epoch-Based Key Rotation . . . . .                    | 7        |
| <b>6</b> | <b>Empirical Evaluation</b>                           | <b>7</b> |
| 6.1      | Benchmarks . . . . .                                  | 7        |
| 6.2      | Geographic Distribution . . . . .                     | 7        |
| 6.3      | Comparison with Classical Threshold Schemes . . . . . | 7        |
| <b>7</b> | <b>Related Work</b>                                   | <b>8</b> |
| <b>8</b> | <b>Conclusion</b>                                     | <b>8</b> |

# 1 Introduction

Threshold signatures enable a group of  $n$  parties to collaboratively sign messages such that any subset of  $t$  or more parties can produce a valid signature, but fewer than  $t$  parties cannot. In blockchain consensus, threshold signatures aggregate validator votes into compact block certificates, reducing verification from  $O(n)$  individual signature checks to a single aggregate verification.

Classical threshold schemes—BLS threshold [2], FROST [3], and CGGMP21 [4]—rely on discrete logarithm or pairing assumptions vulnerable to Shor’s algorithm [1]. Post-quantum threshold signatures based on lattice assumptions remain an active area of research, with practical schemes lagging behind classical alternatives.

Ringtail addresses this gap with a practical lattice-based threshold signature scheme optimized for blockchain consensus. The design prioritizes:

- **Low round complexity:** 2 rounds (1 message-independent precomputation + 1 signing).
- **Moderate signature size:**  $\sim 56$  KB, acceptable for block headers.
- **Fast verification:** Single matrix-vector multiplication.
- **Standard assumptions:** Module-LWE with conservative parameters.

## Contributions.

1. The Ringtail threshold signature scheme over Module-LWE (§3).
2. EUF-CMA security proof reducing to Module-LWE hardness (§4).
3. Practical parameter selection with concrete security estimates (§??).
4. Integration with Lux Quasar dual-certificate consensus (§5).
5. Benchmarks: 0.6s signing,  $< 1$ ms verification (§6).

# 2 Preliminaries

## 2.1 Polynomial Rings

Let  $R = \mathbb{Z}[X]/(X^N + 1)$  and  $R_q = \mathbb{Z}_q[X]/(X^N + 1)$  where  $N$  is a power of 2 and  $q$  is a prime with  $q \equiv 1 \pmod{2N}$  (enabling NTT). Elements of  $R_q$  are polynomials of degree  $< N$  with coefficients in  $\mathbb{Z}_q$ .

**Definition 2.1** (Module-LWE). *The Module-LWE problem with parameters  $(N, q, M, N', \sigma)$  is: given  $\mathbf{A} \xleftarrow{\$} R_q^{M \times N'}$  and  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$  where  $\mathbf{s} \xleftarrow{\$} D_{\sigma}^{N'}$  and  $\mathbf{e} \xleftarrow{\$} D_{\sigma}^M$  (discrete Gaussian with parameter  $\sigma$ ), distinguish  $(\mathbf{A}, \mathbf{b})$  from  $(\mathbf{A}, \mathbf{u})$  where  $\mathbf{u} \xleftarrow{\$} R_q^M$ .*

## 2.2 Shamir Secret Sharing over Rings

**Definition 2.2** (Ring-Based Shamir Sharing). *Given a secret vector  $\mathbf{s} \in R_q^{N'}$ , we share each component  $s_j$  using a polynomial  $f_j(X) = s_j + \sum_{i=1}^{t-1} a_{j,i}X^i$  where  $a_{j,i} \xleftarrow{\$} R_q$ . Party  $i$  receives share  $\mathbf{s}_i = (f_1(i), f_2(i), \dots, f_{N'}(i)) \in R_q^{N'}$ .*

Lagrange interpolation over  $R_q$  enables reconstruction from any  $t$  shares:

$$\mathbf{s} = \sum_{i \in S} \lambda_i \cdot \mathbf{s}_i, \quad \lambda_i = \prod_{j \in S, j \neq i} \frac{j}{j - i} \in R_q \quad (1)$$

| Parameter                | Symbol     | Value                                   |
|--------------------------|------------|-----------------------------------------|
| Ring dimension           | $N$        | 256                                     |
| Modulus                  | $q$        | $2^{48} + 2561$ ( $= 0x1000000004A01$ ) |
| Module rows              | $M$        | 8                                       |
| Module columns           | $N'$       | 7                                       |
| Gaussian width (key gen) | $\sigma_e$ | 6.108                                   |
| Gaussian width (signing) | $\sigma^*$ | $2^{37.33}$                             |
| Rejection bound          | $B$        | $2^{48.61}$                             |
| Challenge ternary weight | $\kappa$   | 23                                      |
| Rounding parameter       | $\xi$      | 30                                      |

Table 1: Ringtail parameters

## 2.3 Ringtail Parameters

# 3 The Ringtail Scheme

## 3.1 Key Generation (Trusted Dealer)

---

### Algorithm 1 Ringtail Key Generation

---

**Require:** Threshold  $t$ , parties  $n$ , parameters  $(N, q, M, N', \sigma_e)$

---

```

1:  $\mathbf{A} \xleftarrow{\$} R_q^{M \times N'}$  ▷ public matrix
2:  $\mathbf{s} \xleftarrow{\$} D_{\sigma_e}^{N'}$  ▷ secret key
3:  $\mathbf{e} \xleftarrow{\$} D_{\sigma_e}^M$  ▷ error
4:  $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$  ▷ public key component
5:  $\tilde{\mathbf{b}} \leftarrow \text{Round}(\mathbf{b}, \xi)$  ▷ rounded for verification
6:  $\{\mathbf{s}_1, \dots, \mathbf{s}_n\} \leftarrow \text{ShamirShare}(\mathbf{s}, t, n)$  ▷ secret shares
7: ▷ Generate correlated randomness for signing
8: for each pair  $(i, j)$  with  $i < j$  do
9:    $\text{seed}_{i,j} \xleftarrow{\$} \{0, 1\}^{256}$  ▷ shared PRF seed
10:   Send  $\text{seed}_{i,j}$  to parties  $i$  and  $j$ 
11: end for
12: ▷ Generate MAC keys for authentication
13: for each pair  $(i, j)$  do
14:    $\text{MACkey}_{i \rightarrow j} \xleftarrow{\$} \{0, 1\}^{256}$ 
15: end for
16: return  $\text{pk} = (\mathbf{A}, \tilde{\mathbf{b}})$ , shares  $\{\mathbf{s}_i, \text{seeds}_i, \text{MACkeys}_i\}$ 

```

---

### 3.2 Signing Protocol (2 Rounds)

---

**Algorithm 2** Ringtail Signing — Round 1 (Message-Independent)

---

**Require:** Party  $i$ , PRF key, signers set  $\mathcal{S}$  with  $|\mathcal{S}| \geq t$

```

1:                                     ▷ Generate party  $i$ 's randomness share
2:  $\mathbf{r}_i \leftarrow \text{PRF}(\text{key}, \text{session} \| i)$                                      ▷ deterministic from seed
3: for each party  $j \in \mathcal{S} \setminus \{i\}$  do
4:    $\mathbf{r}_{i,j} \leftarrow \text{PRF}(\text{seed}_{i,j}, \text{session})$                                      ▷ correlated randomness
5: end for
6:  $\mathbf{y}_i \leftarrow \mathbf{r}_i + \sum_{j:j>i} \mathbf{r}_{i,j} - \sum_{j:j<i} \mathbf{r}_{j,i}$                                      ▷ combined randomness with cancellation
7:  $\mathbf{w}_i \leftarrow \mathbf{A}\mathbf{y}_i$                                      ▷ commitment
8:                                     ▷ Compute MAC for authentication
9: for each  $j \in \mathcal{S} \setminus \{i\}$  do
10:    $\text{mac}_{i \rightarrow j} \leftarrow \text{HMAC}(\text{MACkey}_{i \rightarrow j}, \mathbf{w}_i)$ 
11: end for
12: broadcast  $(\mathbf{w}_i, \{\text{mac}_{i \rightarrow j}\}_{j \in \mathcal{S}})$ 

```

---



---

**Algorithm 3** Ringtail Signing — Round 2 (Message-Dependent)

---

**Require:** Party  $i$ , message  $\mu$ , Round 1 data  $\{(\mathbf{w}_j, \text{mac}_j)\}_{j \in \mathcal{S}}$

```

1:                                     ▷ Verify MACs from all parties
2: for each  $j \in \mathcal{S} \setminus \{i\}$  do
3:   Verify  $\text{mac}_{j \rightarrow i}$  using  $\text{MACkey}_{j \rightarrow i}$ 
4:   abort if verification fails
5: end for
6:  $\mathbf{w} \leftarrow \sum_{j \in \mathcal{S}} \lambda_j \cdot \mathbf{w}_j$                                      ▷ combined commitment
7:  $c \leftarrow H(\mathbf{w}, \mu)$                                      ▷ challenge: ternary polynomial, weight  $\kappa$ 
8:  $\mathbf{z}_i \leftarrow \mathbf{y}_i + c \cdot \lambda_i \cdot \mathbf{s}_i$                                      ▷ partial response
9:                                     ▷ Rejection sampling: abort if  $\|\mathbf{z}_i\|$  too large
10: if  $\|\mathbf{z}_i\|_\infty > B$  then
11:   abort and restart with new session
12: end if
13: broadcast  $\mathbf{z}_i$ 

```

---

### 3.3 Finalization and Verification

The combiner (any party or external verifier) assembles the signature:

$$\mathbf{z} = \sum_{i \in \mathcal{S}} \lambda_i \cdot \mathbf{z}_i$$

Verification checks:

1.  $\|\mathbf{z}\|_\infty \leq t \cdot B$  (response is small).
2. Recompute  $\mathbf{w}' = \mathbf{A}\mathbf{z} - c \cdot \tilde{\mathbf{b}}$  and check  $c = H(\text{Round}(\mathbf{w}', \xi), \mu)$ .

## 4 Security Analysis

### 4.1 Unforgeability

**Theorem 4.1** (EUF-CMA Security). *Ringtail is existentially unforgeable under chosen-message attacks in the random oracle model, assuming the hardness of Module-LWE( $N, q, M, N', \sigma_e$ ).*

Concretely, any adversary making  $Q_H$  hash queries and  $Q_S$  signing queries has advantage:

$$\text{Adv}^{\text{EUF-CMA}} \leq Q_H \cdot \text{Adv}^{\text{MLWE}} + \frac{Q_S}{2^{128}}$$

*Proof.* We proceed through a sequence of games.

**Game 0:** The real EUF-CMA game. The adversary controls  $t - 1$  parties and interacts with the signing oracle.

**Game 1:** Replace the public key  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$  with a uniformly random  $\mathbf{u} \xleftarrow{\$} R_q^M$ . The distinguishing advantage between Games 0 and 1 is  $\text{Adv}^{\text{MLWE}}$  by definition.

**Game 2:** In Game 1, the public key is independent of  $\mathbf{s}$ . We reprogram the random oracle  $H$  to extract the forgery. When the adversary queries  $H(\mathbf{w}^*, \mu^*)$  for a fresh  $(\mathbf{w}^*, \mu^*)$ , the challenger programs  $c^* \leftarrow H(\mathbf{w}^*, \mu^*)$  and checks if the adversary can produce a valid  $\mathbf{z}^*$  satisfying:

$$\mathbf{A}\mathbf{z}^* - c^* \cdot \tilde{\mathbf{b}} \approx \mathbf{w}^*$$

Given  $\tilde{\mathbf{b}} = \mathbf{u}$  (random), this requires finding short  $\mathbf{z}^*$  such that  $\mathbf{A}\mathbf{z}^* \approx \mathbf{w}^* + c^* \cdot \mathbf{u}$ , which is an instance of the SIS (Short Integer Solution) problem—known to be hard under Module-LWE.

The probability of a collision in  $H$  (two queries yielding the same challenge for different messages) is  $\leq Q_H^2/|C|$  where  $|C| = \binom{N}{\kappa} 2^\kappa$  is the challenge space, negligible for  $\kappa = 23$  and  $N = 256$ .

Combining the game transitions:  $\text{Adv}^{\text{Game 0}} \leq \text{Adv}^{\text{MLWE}} + \text{Adv}^{\text{Game 2}} \leq \text{Adv}^{\text{MLWE}} + Q_H/|C|$ .  $\square$

## 4.2 Threshold Security

**Lemma 4.2** (Threshold Unforgeability). *An adversary controlling  $t - 1$  parties and observing  $Q_S$  signing sessions cannot forge a signature on a new message. The adversary's view in each signing session is statistically close to a simulation that does not use the honest parties' shares, by the zero-knowledge property of the commitment scheme.*

*Proof.* In each signing session, the adversary sees the honest parties' commitments  $\mathbf{w}_i = \mathbf{A}\mathbf{y}_i$  and responses  $\mathbf{z}_i = \mathbf{y}_i + c\lambda_i\mathbf{s}_i$ . Given  $\mathbf{w}_i$  and  $\mathbf{z}_i$ , the adversary can compute  $c\lambda_i\mathbf{s}_i = \mathbf{z}_i - \mathbf{y}_i$ . However, reconstructing  $\mathbf{y}_i$  from  $\mathbf{w}_i = \mathbf{A}\mathbf{y}_i$  is an instance of SIS (finding a short preimage of  $\mathbf{w}_i$  under  $\mathbf{A}$ ), which is hard under Module-LWE.

The rejection sampling ensures that  $\mathbf{z}_i$  is statistically independent of  $\mathbf{s}_i$ : the distribution of  $\mathbf{z}_i$  conditioned on any  $\mathbf{s}_i$  is within statistical distance  $2^{-128}$  of the distribution conditioned on any other  $\mathbf{s}_i$ , by the properties of discrete Gaussian rejection sampling with parameter  $\sigma^* \gg \|c\lambda_i\mathbf{s}_i\|$ .

Therefore,  $t - 1$  parties' shares reveal no information about the  $t$ -th share (by Shamir sharing), and the signing transcripts reveal no additional information about the secret key (by the zero-knowledge simulation).  $\square$

## 4.3 Concrete Security Estimates

**Proposition 4.3** (Security Level). *With parameters  $N = 256$ ,  $q \approx 2^{48}$ ,  $M = 8$ ,  $N' = 7$ ,  $\sigma_e = 6.1$ : the Module-LWE instance has root Hermite factor  $\delta \approx 1.0045$ , corresponding to  $> 128$ -bit classical security and  $> 100$ -bit quantum security against known lattice algorithms (BKZ 2.0, sieving).*

*Proof.* The LWE dimension is  $d = M \cdot N = 8 \times 256 = 2048$ . The noise ratio is  $\alpha = \sigma_e \sqrt{2\pi}/q \approx 15.3/2^{48} \approx 2^{-44.1}$ .

The best known attack (primal lattice attack via BKZ with sieving) has cost:

$$T_{\text{classical}} \approx 2^{0.292 \cdot \beta + 16.4}$$

where the BKZ blocksize  $\beta$  satisfies  $\delta^{2\beta-d} \cdot q^{d/\beta} = \sigma_e$ , giving  $\beta \approx 430$  and  $T_{\text{classical}} \approx 2^{142}$ .

For quantum attacks (Grover-enhanced sieving),  $T_{\text{quantum}} \approx 2^{0.265\beta + 16.4} \approx 2^{130}$ .  $\square$

## 5 Integration with Lux Quasar

Ringtail integrates with the Lux Quasar consensus engine as the post-quantum layer of the dual-certificate mechanism:

1. **BLS layer:** Classical BLS12-381 threshold signatures provide fast (1-round) aggregation for the primary finality certificate.
2. **Ringtail layer:** Lattice-based threshold signatures provide quantum-safe finality anchors every 3 seconds (“quantum bundles”).

The dual-certificate design ensures that:

- Under classical adversaries: BLS provides fast finality (500ms).
- Under quantum adversaries: Ringtail provides post-quantum finality (3s bundles).
- The attack window for quantum adversaries is  $< 3s$ , far too short for any known quantum algorithm to break BLS.

### 5.1 Epoch-Based Key Rotation

Ringtail keys are rotated every 10–60 minutes via epoch transitions. Each epoch generates new Shamir shares for the current validator set, with the previous epoch’s keys preserved for cross-epoch signature verification.

## 6 Empirical Evaluation

### 6.1 Benchmarks

| Operation                  | Time  | Size                 |
|----------------------------|-------|----------------------|
| Key generation (3 parties) | 1.2s  | 168 KB (3 shares)    |
| Round 1 (per party)        | 85ms  | 14.3 KB (commitment) |
| Round 2 (per party)        | 120ms | 14.3 KB (response)   |
| Total signing (2-of-3)     | 0.6s  | —                    |
| Verification               | 0.8ms | —                    |
| Signature size             | —     | 56.2 KB              |
| Public key size            | —     | 57.4 KB              |

Table 2: Ringtail performance benchmarks

### 6.2 Geographic Distribution

With 5 parties across US-East, US-West, EU-West, Asia-Pacific, and South America (100–250ms inter-region latency), the end-to-end signing time was 2.5 seconds for 3-of-5 threshold, well within the 3-second quantum bundle interval.

### 6.3 Comparison with Classical Threshold Schemes

Ringtail’s signature is  $\sim 1000\times$  larger than classical schemes, but this is acceptable for block headers (which carry one signature per block). The signing time is dominated by network latency rather than computation, and the 0.8ms verification time is competitive with classical schemes.

| Scheme          | Sig Size     | Sign Time    | Verify       | PQ Safe    |
|-----------------|--------------|--------------|--------------|------------|
| BLS Threshold   | 48 B         | 5ms          | 2ms          | No         |
| FROST           | 64 B         | 10ms         | 1ms          | No         |
| CGGMP21         | 65 B         | 200ms        | <1ms         | No         |
| <b>Ringtail</b> | <b>56 KB</b> | <b>600ms</b> | <b>0.8ms</b> | <b>Yes</b> |

Table 3: Threshold signature scheme comparison

## 7 Related Work

Bendlin et al. [5] introduced lattice-based threshold signatures using NTRU. Boneh et al. [6] proposed threshold signatures from lattice assumptions but with impractical parameters. Recent works by Cozzo and Smart [7] and Espitau et al. [8] have improved practical lattice-based threshold schemes. Ringtail builds on these foundations with optimizations specific to blockchain consensus: deterministic correlated randomness, MAC-based authentication for malicious security, and the rejection sampling parameters tuned for the NTT-friendly ring structure of  $R_q$ .

## 8 Conclusion

Ringtail provides the first practical lattice-based threshold signature scheme for blockchain consensus, achieving 0.6-second signing and sub-millisecond verification with  $> 128$ -bit classical security. The two-round protocol integrates naturally with the Lux Quasar dual-certificate mechanism, providing quantum-safe finality anchors without impacting the BLS fast path. Ringtail’s parameters are derived from the Module-LWE assumption with concrete security estimates, and the scheme is implemented in production on the Lux network.

## References

- [1] P. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Computing*, 1997.
- [2] A. Boldyreva, “Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme,” in *PKC*, 2003.
- [3] C. Komlo and I. Goldberg, “FROST: Flexible round-optimized Schnorr threshold signatures,” in *SAC*, 2020.
- [4] R. Canetti et al., “UC non-interactive, proactive, threshold ECDSA with identifiable aborts,” in *CCS*, 2021.
- [5] R. Bendlin, S. Krehbiel, C. Peikert, and A. Rosen, “Semi-homomorphic encryption and multiparty computation,” in *EUROCRYPT*, 2013.
- [6] D. Boneh et al., “Threshold cryptosystems from threshold fully homomorphic encryption,” in *CRYPTO*, 2018.
- [7] D. Cozzo and N. Smart, “Sharing the LUOV: Threshold post-quantum signatures,” in *IMA*, 2023.
- [8] T. Espitau, M. Tibouchi, and A. Wallet, “Practical Fiat-Shamir with aborts threshold signatures from lattices,” 2024.



- [9] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *JACM*, 2009.